Experiment No : 01

Aim : Learn basics of Numpy library for storing and efficiently processing any external data into python execution pipeline.

Theory : NumPy (short for Numerical Python) provides an efficient interface to store and operate on dense data buffers. In some ways, NumPy arrays are like Python's built-in list type, but NumPy arrays provide much more efficient storage and data operations as the arrays grow larger in size. NumPy arrays form the core of nearly the entire ecosystem of data science tools in Python

NumPy is a Python library used for working with arrays. It also has functions for working in domain of linear algebra, fourier transform, and matrices. NumPy was created in 2005 by Travis Oliphant. It is an open source project and you can use it freely.

In Python we have lists that serve the purpose of arrays, but they are slow to process. NumPy aims to provide an array object that is up to 50x faster than traditional Python lists. The array object in NumPy is called ndarray, it provides a lot of supporting functions that make working with ndarray very easy. Arrays are very frequently used in data science, where speed and resources are very important.

NumPy arrays are stored at one continuous place in memory unlike lists, so processes can access and manipulate them very efficiently. This behavior is called locality of reference in computer science. This is the main reason why NumPy is faster than lists. Also it is optimized to work with latest CPU architectures.

A Python List Is More Than Just a List

Let's consider now what happens when we use a Python data structure that holds many Python objects. The standard mutable multi-element container in Python is the list. We can create a list of integers as follows:

```
import numpy
numpy.__version__
```

```
'2.0.2'
```

```
L = list(range(10))
L
```

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
type(L[0])
```

```
int
```

```
L2 = [str(c) for c in L]
L2
```

```
['0', '1', '2', '3', '4', '5', '6', '7', '8', '9']
```

```
type(L2[0])
```

```
str
```

```
L3 = [True, "2", 3.0, 4]
[type(item) for item in L3]
```

```
[bool, str, float, int]
```

Fixed-Type Arrays in Python

Python offers several different options for storing data in efficient, fixed-type data buffers. The built-in array module (available since Python 3.3) can be used to create dense arrays of a uniform type:

```
import array
L = list(range(10))
A = array.array('i', L)
A
```

```
array('i', [0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

Creating Arrays from Python Lists

First, we can use np.array to create arrays from Python lists:

```
import numpy as np
# integer array:
np.array([1, 4, 2, 5, 3])
```

```
array([1, 4, 2, 5, 3])
```

Remember that unlike Python lists, NumPy is constrained to arrays that all contain the same type. If types do not match, NumPy will upcast if possible (here, integers are up-cast to floating point):

```
import numpy as np
np.array([3.14, 4, 2, 3])
```

```
array([3.14, 4.  , 2.  , 3.  ])
```

If we want to explicitly set the data type of the resulting array, we can use the dtype keyword:

```
np.array([1, 2, 3, 4], dtype='float32')
```

```
array([1., 2., 3., 4.], dtype=float32)
```

Finally, unlike Python lists, NumPy arrays can explicitly be multi-dimensional; here's one way of initializing a multidimensional array using a list of lists:

```
# nested lists result in multi-dimensional arrays
np.array([range(i,i + 3) for i in [2, 4, 6]])
```

```
array([[2, 3, 4],
       [4, 5, 6],
       [6, 7, 8]])
```

Creating Arrays from Scratch

Especially for larger arrays, it is more efficient to create arrays from scratch using routines built into NumPy. Here are several examples:

```
# Create a length-10 integer array filled with zeros
np.zeros(10, dtype=int)
```

```
array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0])
```

```
# Create a 3x5 floating-point array filled with ones
np.ones((3, 5), dtype=float)
```

```
array([[1., 1., 1., 1., 1.],
       [1., 1., 1., 1., 1.],
       [1., 1., 1., 1., 1.]])
```

```
np.full((3, 5), 3.14)
```

```
array([[3.14, 3.14, 3.14, 3.14, 3.14],
       [3.14, 3.14, 3.14, 3.14, 3.14],
       [3.14, 3.14, 3.14, 3.14, 3.14]])
```

```
# Create an array filled with a linear sequence
# Starting at 0, ending at 20, stepping by 2
# (this is similar to the built-in range() function)
np.arange(0, 20, 2)
```

```
array([ 0,  2,  4,  6,  8, 10, 12, 14, 16, 18])
```

```
# Create an array of five values evenly spaced between 0 and 1
np.linspace(0, 1, 5)
```

```
array([0.  , 0.25, 0.5 , 0.75, 1.  ])
```

```
# Create a 3x3 array of uniformly distributed
# random values between 0 and 1
np.random.random((3, 3))
```

```
array([[0.87144426, 0.45672693, 0.92704979],
       [0.63068438, 0.76643661, 0.45425838],
       [0.88790115, 0.89582469, 0.89965742]])
```

```
# Create a 3x3 array of normally distributed random values
# with mean 0 and standard deviation 1
np.random.normal(0, 1, (3, 3))
```

```
array([[-1.17891436,  0.8668381 , -0.99073694],
       [-1.47592182, -0.84973169, -1.08656465],
       [ 0.65012243,  1.28097653, -0.21485491]])
```

```
# Create a 3x3 array of random integers in the interval [0, 10)
np.random.randint(0, 10, (3, 3))
```

```
array([[8, 7, 4],
       [7, 0, 9],
       [3, 7, 3]])
```

```
# Create a 3x3 identity matrix
np.eye(5)
```

```
array([[1., 0., 0., 0., 0.],
       [0., 1., 0., 0., 0.],
       [0., 0., 1., 0., 0.],
       [0., 0., 0., 1., 0.],
       [0., 0., 0., 0., 1.]])
```

NumPy Standard Data Types

NumPy arrays contain values of a single type, so it is important to have detailed knowledge of those types and their limitations. Because NumPy is built in C, the types will be familiar to users of C, Fortran, and other related languages.

The standard NumPy data types are listed in the following table. Note that when constructing an array, they can be specified using a string:

np.zeros(10, dtype='int16')

Or using the associated NumPy object:

np.zeros(10, dtype=np.int16)

NumPy Array Attributes

We will learn about important Attributes with NumPy Array objects

Each array object has attributes ndim (the number of dimensions), shape (the size of each dimension), and size (the total size of the array), the dtype, the data type of the array :

```
#Consider following sample arrays
np.random.seed(0)  # seed for reproducibility

x1 = np.random.randint(10, size=6)  # One-dimensional array
x2 = np.random.randint(10, size=(3, 4))  # Two-dimensional array
x3 = np.random.randint(10, size=(3, 4, 5))  # Three-dimensional
```

```
print("x3 ndim: ", x3.ndim)
print("x3 shape:", x3.shape)
print("x3 size: ", x3.size)
print("dtype:", x3.dtype)
```

```
x3 ndim:  3
x3 shape: (3, 4, 5)
x3 size:  60
dtype: int64
```

Other attributes include itemsize, which lists the size (in bytes) of each array element, and nbytes, which lists the total size (in bytes) of the array:

```
print("itemsize:", x3.itemsize, "bytes")
print("nbytes:", x3.nbytes, "bytes")
```

```
itemsize: 8 bytes
nbytes: 480 bytes
```

Array Indexing: Accessing Single Elements

Next we learn how to access single element in a NumPy array NumPy follows indexing similar to that of Python in a dimension index starts at 0 till length-1

So x1[0] will mean 0th element and x1[5] means sixth element in array x1.

We can use negative index value to indicate accessing elements from back side of array.

In a multi-dimensional array, items can be accessed using a comma-separated tuple of indices as shown in below code cell.

```
# acessing third list's first element
x2[2, 0]
```

```
np.int64(1)
```

```
# accessing second last element from second list
x2[1, -2]
```

```
np.int64(8)
```

```
# modifying value at a perticular index
x2[0, 0] = 12
x2
```

```
array([[12,  5,  2,  4],
       [ 7,  6,  8,  8],
       [ 1,  6,  7,  7]])
```

Array Slicing: Accessing Subarrays

Just as we can use square brackets to access individual array elements, we can also use them to access subarrays with the slice notation, marked by the colon (:) character. The NumPy slicing syntax follows that of the standard Python list; to access a slice of an array x, use this:

x[start:stop:step]

If any of these are unspecified, they default to the values start=0, stop=size of dimension, step=1. We'll take a look at accessing sub-arrays in one dimension and in multiple dimensions.

One-dimensional subarrays

```
x = np.arange(10)
x
```

```
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
x[:5]  # first five elements
```

```
array([0, 1, 2, 3, 4])
```

```
x[:5]  # elements after index five
```

```
array([0, 1, 2, 3, 4])
```

```
x[4:7]  # sub-array of index 4, 5, 6
```

```
array([4, 5, 6])
```

```
x[::2]  # every other element
```

```
array([0, 2, 4, 6, 8])
```

```
x[2::2]  # every other element starting at index 2
```

```
array([2, 4, 6, 8])
```

A potentially confusing case is when the step value is negative. In this case, the defaults for start and stop are swapped. This becomes a convenient way to reverse an array:

```
x[::-1]  # all elements, reversed
```

```
array([9, 8, 7, 6, 5, 4, 3, 2, 1, 0])
```

```
x[3::-2]  # reversed every other from index 3
```

```
array([3, 1])
```

Multi-dimensional subarrays

Multi-dimensional slices work in the same way, with multiple slices separated by commas. For example:

```
x2[:2, :3]  # This is sub Array of x2 with first two rows and first three columns
```

```
array([[12,  5,  2],
       [ 7,  6,  8]])
```

```
#Check how we can reverse the multidimension array
x2[::-1, ::-1]
```

```
array([[ 7,  7,  6,  1],
       [ 8,  8,  6,  7],
       [ 4,  2,  5, 12]])
```

Accessing array rows and columns

One commonly needed routine is accessing of single rows or columns of an array. This can be done by combining indexing and slicing, using an empty slice marked by a single colon (:):

```
print(x2[:, 0])  # first column of x2
```

```
[12  7  1]
```

```
print(x2[0, :])  # first row of x2
```

```
[12  5  2  4]
```

Subarrays as no-copy views

One important–and extremely useful–thing to know about array slices is that they return views rather than copies of the array data. This is one area in which NumPy array slicing differs from Python list slicing: in lists, slices will be copies. Consider our two-dimensional array from before:

```
#Extract 2*2 sub array from x2
x2_sub = x2[:2, :2]
print(x2_sub)
```

```
[[12  5]
 [ 7  6]]
```

```
x2_sub[0, 0] = 99
#Above statement not oly modifies subarray but also the original array as well
print(x2_sub)
```

```
[[99  5]
 [ 7  6]]
```

Despite the nice features of array views, it is sometimes useful to instead explicitly copy the data within an array or a subarray. This can be most easily done with the copy() method:

```
x2_sub_copy = x2_sub.copy()
x2_sub_copy[0, 0] = 42
print(x2_sub_copy)
print(x2)
```

```
[[42  5]
 [ 7  6]]
[[99  5  2  4]
 [ 7  6  8  8]
 [ 1  6  7  7]]
```

Array Concatenation and Splitting

All of the preceding routines worked on single arrays. It's also possible to combine multiple arrays into one, and to conversely split a single array into multiple arrays. We'll take a look at those operations here.

Concatenation of arrays

Concatenation, or joining of two arrays in NumPy, is primarily accomplished using the routines np.concatenate, np.vstack, and np.hstack. np.concatenate takes a tuple or list of arrays as its first argument, as we can see here:

```
x = np.array([1, 2, 3])
y = np.array([3, 2, 1])
np.concatenate([x, y])
```

```
array([1, 2, 3, 3, 2, 1])
```

```
grid = np.array([[1, 2, 3],[4, 5, 6]])
```

```
# concatenate along the first axis
np.concatenate([grid, grid])
```

```
array([[1, 2, 3],
       [4, 5, 6],
       [1, 2, 3],
       [4, 5, 6]])
```

```
# concatenate along the second axis (zero-indexed)
np.concatenate([grid, grid], axis=1)
```

```
array([[1, 2, 3, 1, 2, 3],
       [4, 5, 6, 4, 5, 6]])
```

When joining arrays of mixed dimensions, it can be clearer to use the np.vstack (vertical stack) and np.hstack (horizontal stack) functions as shown below:

```
x = np.array([1, 2, 3])
grid = np.array([[9, 8, 7],
                 [6, 5, 4]])

# vertically stack the arrays
np.vstack([x, grid])
```

```
array([[1, 2, 3],
       [9, 8, 7],
       [6, 5, 4]])
```

```
# horizontally stack the arrays
y = np.array([[99],
              [99]])
np.hstack([grid, y])
```

```
array([[ 9,  8,  7, 99],
       [ 6,  5,  4, 99]])
```

Splitting of arrays

The opposite of concatenation is splitting, which is implemented by the functions np.split, np.hsplit, and np.vsplit. For each of these, we can pass a list of indices giving the split points:

```
x = [1, 2, 3, 99, 99, 3, 2, 1]
x1, x2, x3 = np.split(x, [3, 5])
print(x1, x2, x3)
```

```
[1 2 3] [99 99] [3 2 1]
```

```
import numpy as np
grid = np.arange(16).reshape((4, 4))
grid
```

```
array([[ 0,  1,  2,  3],
       [ 4,  5,  6,  7],
       [ 8,  9, 10, 11],
       [12, 13, 14, 15]])
```

```
upper, lower = np.vsplit(grid, [2])
print(upper)
print(lower)
```

```
[[0 1 2 3]
 [4 5 6 7]]
[[ 8  9 10 11]
 [12 13 14 15]]
```

```
left, right = np.hsplit(grid, [2])
print(left)
print(right)
```

```
[[ 0  1]
 [ 4  5]
 [ 8  9]
 [12 13]]
[[ 2  3]
 [ 6  7]
 [10 11]
 [14 15]]
```

Introducing UFuncs

For many types of operations, NumPy provides a convenient interface into just this kind of statically typed, compiled routine. This is known as a vectorized operation. This can be accomplished by simply performing an operation on the array, which will then be applied to each element. This vectorized approach is designed to push the loop into the compiled layer that underlies NumPy, leading to much faster execution.

```
#Consider following loop based implementation to ind reciprocals for each element of an array
np.random.seed(0)

def compute_reciprocals(values):
    output = np.empty(len(values))
    for i in range(len(values)):
        output[i] = 1.0 / values[i]
    return output

big_array = np.random.randint(1, 100, size=1000000)
%timeit compute_reciprocals(big_array)
```

```
2.37 s ± 423 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)
```

```
#Same operatoin using UFuncs applying '/' over array elements
%timeit (1.0 / big_array)
```

```
2.16 ms ± 147 µs per loop (mean ± std. dev. of 7 runs, 1000 loops each)
```

Vectorized operations in NumPy are implemented via ufuncs, whose main purpose is to quickly execute repeated operations on values in NumPy arrays. UFuncs are extremely flexible – before we saw an operation between a scalar and an array, but we can also operate between two arrays as well as multidimentional arrays.

Computations using vectorization through ufuncs are nearly always more efficient than their counterpart implemented using Python loops, especially as the arrays grow in size. Any time you see such a loop in a Python script, you should consider whether it can be replaced with a vectorized expression.

Exploring NumPy's UFuncs

Ufuncs exist in two flavors: unary ufuncs, which operate on a single input, and binary ufuncs, which operate on two inputs.

Array arithmetic

NumPy's ufuncs feel very natural to use because they make use of Python's native arithmetic operators. The standard addition, subtraction, multiplication, and division can all be used:

```
x = np.arange(4)
print("x     =", x)
print("x + 5 =", x + 5)
print("x - 5 =", x - 5)
print("x * 2 =", x * 2)
print("x / 2 =", x / 2)
print("x // 2 =", x // 2)  # floor division
#some advanced arithmatic expression   -(x/2+1)^2
-(0.5*x + 1) ** 2
```

```
x     = [0 1 2 3]
x + 5 = [5 6 7 8]
x - 5 = [-5 -4 -3 -2]
x * 2 = [0 2 4 6]
x / 2 = [0.  0.5 1.  1.5]
x // 2 = [0 0 1 1]
array([-1.  , -2.25, -4.  , -6.25])
```

Specialized ufuncs

NumPy has many more ufuncs available, including hyperbolic trig functions, bitwise arithmetic, comparison operators, conversions from radians to degrees, rounding and remainders, and much more. A look through the NumPy documentation reveals a lot of interesting functionality.

Another excellent source for more specialized and obscure ufuncs is the submodule scipy.special. If you want to compute some obscure mathematical function on your data, chances are it is implemented in scipy.special. There are far too many functions to list them all, but the following snippet shows a couple that might come up in a statistics context:

```
#importing package special from scipy package
```

```
from scipy import special


# Gamma functions (generalized factorials) and related functions
x = [1, 5, 10]
print("gamma(x)     =", special.gamma(x))
print("ln|gamma(x)| =", special.gammaln(x))
print("beta(x, 2)   =", special.beta(x, 2))
```

```
gamma(x)     = [1.0000e+00 2.4000e+01 3.6288e+05]
ln|gamma(x)| = [ 0.          3.17805383 12.80182748]
beta(x, 2)   = [0.5        0.03333333 0.00909091]
```

Aggregates

For binary ufuncs, there are some interesting aggregates that can be computed directly from the object. For example, if we'd like to reduce an array with a particular operation, we can use the reduce method of any ufunc. A reduce repeatedly applies a given operation to the elements of an array until only a single result remains.

For example, calling reduce on the add ufunc returns the sum of all elements in the array:

```
x = np.arange(1, 6)
np.add.reduce(x)
```

```
np.int64(15)
```

```
np.multiply.reduce(x)
```

```
np.int64(352836)
```

```
#note the difference in output with accumulate
np.add.accumulate(x)
```

```
array([  1,   3,   6, 105, 204, 207, 209, 210])
```

Q.1 What are UFuncs in numpy?

NumPy ufuncs (universal functions) are highly optimized, compiled C functions that perform element-wise operations on NumPy arrays. They are a core feature of NumPy used to implement vectorization, which is significantly faster than using standard Python loops, especially for large datasets. Key Characteristics

```
Vectorization: Ufuncs apply operations to entire arrays at once, pushing the loops into the underlying, efficient C code.

Element-wise operation: A ufunc acts on corresponding elements of input arrays to produce a new output array.

Broadcasting: Ufuncs automatically support broadcasting, a mechanism that allows them to operate on arrays of different shapes ar

Type Casting: They handle automatic data type conversion internally, or allow users to specify the output data type using the dty
```

Common Ufuncs and Methods NumPy includes over 60 built-in ufuncs covering a wide variety of mathematical, logical, and bitwise operations. These can be broadly categorized as:

```
Unary Ufuncs: Operate on a single input array (e.g., np.abs, np.sin, np.sqrt).
Binary Ufuncs: Operate on two input arrays (e.g., np.add, np.multiply, np.maximum).
```

Infix operators (like +, *, /) are actually wrappers for these underlying ufuncs (e.g., a + b calls np.add(a, b)). Ufuncs also provide powerful methods for array reduction and accumulation:

```
reduce: Repeatedly applies a given operation along a specified axis of an array until a single result remains (e.g., np.add.reduc

accumulate: Computes the intermediate results of a reduce operation, returning an array of the same shape as the input (e.g., a c

outer: Computes the operation for all pairs of elements from two different inputs to produce an outer product.

at: Performs unbuffered in-place operations on specific elements specified by indices.
```

Q.2 Which are various attributes of numy arrays object?

NumPy array objects (instances of the ndarray class) have several key attributes that provide information about their structure, size, and the type of data they hold. The most commonly used attributes include:

```
ndim: The number of dimensions (axes) of the array.
shape: A tuple of integers that indicates the size of the array along each dimension. For a 2x3 array, the shape is (2, 3).
size: The total number of elements in the array, which is the product of the elements of the shape tuple.
dtype: An object that describes the data type of the elements in the array (e.g., int64, float64, complex128, bool).
itemsize: The size in bytes of each element in the array. For example, an int64 array has an itemsize of 8 bytes.
nbytes: The total number of bytes consumed by all elements of the array. This is equivalent to size * itemsize.
data: A buffer that contains the actual elements of the array in memory.
strides: A tuple of bytes to step in each dimension when traversing an array, which helps determine memory layout.
flags: An object providing information about the memory layout and properties of the array, such as whether the data is contiguou
base: The original object from which the array was created if it is a view (a shallow copy) of another array. If the array owns i
T: The transposed view of the array, a shorthand for ndarray.transpose().
```

Q.3 If you have 3 dimentional array in numpy object Obj how to identify its size, type and dimentions ?

Assume:

import numpy as np Obj = np.array([[[1,2],[3,4]], [[5,6],[7,8]]])

1️⃣ Find the Shape (Dimensions) Obj.shape

Output:

(2, 2, 2)

This means: 2 blocks × 2 rows × 2 columns

.shape tells you the size along each axis.

2️⃣ Find the Number of Dimensions Obj.ndim

Output:

3

.ndim gives the number of axes (3 ⟶ 3D array).

3️⃣ Find the Total Number of Elements (Size) Obj.size

Output:

8

.size = total number of elements in the array.

4️⃣ Find the Data Type Obj.dtype

Output:

dtype('int64')

.dtype tells you the type of elements stored in the array.

5️⃣ Check the Object Type (Optional) type(Obj)

Output:

<class 'numpy.ndarray'> This confirms that Obj is a NumPy array (ndarray) from NumPy.

Q.4 Consider array object named Obj with dimentions 3*4*2 and you want to convert it to 4*6 shape exlpain how will you do it ? Show this using a code cell, taking an example of 3*4*2 np array of random int between 1 and 50.

To convert a NumPy array from shape (3 × 4 × 2) to (4 × 6), we use the reshape() method.

✅ Key Concept

Reshaping is possible only if the total number of elements remains the same.

Original shape: 3 × 4 × 2 = 24 elements

Target shape: 4 × 6 = 24 elements

Since both have 24 elements, reshaping is valid.

💻 Code Example import numpy as np

Create 3x4x2 array with random integers between 1 and 50

Obj = np.random.randint(1, 51, size=(3, 4, 2))

print("Original Array Shape:", Obj.shape) print(Obj)

Reshape to 4x6

reshaped_Obj = Obj.reshape(4, 6)

print("\nReshaped Array Shape:", reshaped_Obj.shape) print(reshaped_Obj)

🔎 What Happens Internally?

NumPy stores array data in contiguous memory (row-major order by default).

reshape() does not change the data, only how it is viewed.

The elements are reorganized logically without changing their order.

Alternative (Using -1)

You can also let NumPy calculate one dimension automatically:

reshaped_Obj = Obj.reshape(4, -1)

Here, -1 tells NumPy to compute the appropriate size automatically (which becomes 6).

Important Note

If the total number of elements does not match, NumPy will raise:

ValueError: cannot reshape array of size X into shape (...)

Q.5 Consider above 4*6 array, sample 2*3 sub array from left bottom of this array store the result in varible named subObjCpy.

We start from the 4 × 6 reshaped array.

To sample a 2 × 3 sub-array from the bottom-left, we need:

Last 2 rows

First 3 columns

In NumPy slicing:

Last 2 rows → [-2:, :]

First 3 columns → [:, :3]

So combined:

array[-2:, :3]

Q.6 Make changes in 1 perticular value in the subObjCpy. Check values of Obj subObjCpy to check impact of this change of element in original and copied object. What do you conclude?

Since we used .copy() earlier, modifying subObjCpy should NOT affect the original array.

Code Demonstration import numpy as np

Create original 3x4x2 array

Obj = np.random.randint(1, 51, size=(3, 4, 2))

Reshape to 4x6

Obj_4x6 = Obj.reshape(4, 6)

print("Original 4x6 Array:\n", Obj_4x6)

Extract bottom-left 2x3 sub-array (with copy)

subObjCpy = Obj_4x6[-2:, :3].copy()

print("\nOriginal subObjCpy:\n", subObjCpy)

Modify one particular value in subObjCpy

subObjCpy[0, 0] = 999

print("\nModified subObjCpy:\n", subObjCpy)

print("\n4x6 Array After Modification:\n", Obj_4x6)

What You Will Observe

subObjCpy[0, 0] becomes 999

The original Obj_4x6 remains unchanged

Q.7 If you were to filter sub array view from above Obj object and want to apply modification in original and/or sub obj change respective element in other how will you achieve it?

Case 1: Modify Sub-Array → Original Also Changes Code import numpy as np

Create original array

Obj = np.random.randint(1, 51, size=(3, 4, 2))

Reshape to 4x6

Obj_4x6 = Obj.reshape(4, 6)

print("Original 4x6 Array:\n", Obj_4x6)

Create sub-array VIEW (no .copy())

subObjView = Obj_4x6[-2:, :3]

print("\nSub-array View Before Change:\n", subObjView)

Modify sub-array

subObjView[0, 0] = 777

print("\nSub-array View After Change:\n", subObjView)

print("\nOriginal 4x6 Array After Change:\n", Obj_4x6)

What Happens?

Changing subObjView[0,0]

Same element in Obj_4x6 changes automatically

✓ Because subObjView is a view sharing memory

Case 2: Modify Original → Sub-Array Also Changes

Modify original

Obj_4x6[3, 1] = 888

print("\nOriginal After Modification:\n", Obj_4x6) print("\nSub-array View After Original Modification:\n", subObjView)

If [3,1] lies inside the sliced region:

That element also updates in subObjView

Why This Works

NumPy slicing creates a view, meaning:

No new memory is allocated

Both variables reference the same underlying data buffer

You can verify this:

subObjView.base is Obj_4x6

If it returns True, it confirms shared memory.

Conclusion: Thus we have learned basics of Numpy library for storing and efficiently processing any external data into python execution pipeline.